

WAF / CDN Edge-Security Platform

End-to-End Architecture Explained in Simple Terms

Summary · Architecture Diagram · Components · Workflows · Design Trade-offs

The project statement being explained

“Owned end-to-end architecture for a commercial WAF/CDN edge-security platform processing millions of HTTP/HTTPS requests per second across geographically distributed PoPs, sustaining 99.99%+ availability while holding security-processing overhead inside a single-digit-millisecond p99 budget.”

Note: latency numbers, component names and technologies in this document describe a typical, industry-standard design for this kind of platform. They are illustrative reference values, not measurements from any specific company's system.

Contents

(If page numbers show as blank, right-click the table above in Word and choose “Update Field”.)

1. Overall Summary in Simple Terms

The one-paragraph version

Imagine thousands of websites that want protection from hackers and want to load fast for everyone in the world. Instead of each website defending itself, they all send their traffic through our platform first. We run small data centers (called **PoPs – Points of Presence**) in many cities. When anyone opens a protected website, their request goes to the nearest PoP. There, in a few thousandths of a second, we check it for attacks, block anything bad, serve a saved copy if we have one, and forward only clean requests to the real website. I designed how all of those pieces fit together, from the moment a request leaves a user's phone until the reply comes back.

1.1 A simple analogy: the security gate at a stadium

Think of a busy stadium (the customer's website) that millions of people want to enter:

- **Many gates in many places (PoPs):** instead of one entrance, there are gates in every neighbourhood, so nobody has to walk far. Walking distance is network delay.
- **Guards at every gate (WAF / security checks):** they check tickets, scan bags, and stop troublemakers before they ever reach the stadium.
- **A snack stand at the gate (CDN cache):** common items are kept right at the gate, so people do not have to go all the way inside to get them.
- **A head office (control plane):** writes the rules for the guards ("no glass bottles today") and sends the new rules to every gate at once.
- **Security cameras (observability):** record everything so the team can spot new tricks and improve the rules.
- **Backup gates (failover):** if one gate has a problem, people are quietly sent to the next nearest gate.

1.2 What each phrase of the statement means

Phrase	Plain-English meaning
Owned end-to-end architecture	I was responsible for the design of the whole system: how traffic enters, how it is checked, how it is cached, how rules are delivered, how it is monitored and how it survives failures.
Commercial WAF/CDN platform	A product that paying customers use. WAF = Web Application Firewall (blocks web attacks). CDN = Content Delivery Network (stores copies of content close to users so it loads fast).
Edge-security	Security is done at the "edge" of the internet – close to the user – instead of at the customer's own servers.
Millions of HTTP/HTTPS requests per second	Every page, image, or API call is a request. The whole platform handles millions of them every single second, most of them encrypted (HTTPS).

Phrase	Plain-English meaning
Geographically distributed PoPs	Many small data centers spread across countries and continents, each able to do the full job on its own.
99.99%+ availability	The service is up at least 99.99% of the time. That allows only about 52 minutes of downtime in a whole year (about 4.3 minutes a month).
Security-processing overhead	The extra time our security checks add to each request, on top of normal network time.
Single-digit-millisecond p99 budget	For 99 out of every 100 requests, the security checks add less than 10 milliseconds (0.01 seconds). “p99” means the 99th percentile – it measures the slow tail, not just the average.

1.3 The three goals that shaped every design decision

Goal	Target	Why it is hard	Main techniques used
Scale	Millions of requests / second	Traffic spikes 10–100× during attacks or big events	Anycast spreading, many PoPs, stateless servers, kernel-level packet filtering, horizontal scaling
Reliability	≥ 99.99% available	Hardware, networks, software pushes and attacks all fail in different ways	No single point of failure, automatic failover, staged rollouts, fail-safe defaults
Speed	< 10 ms p99 security overhead	Deep inspection of every request is expensive	Compiled rules, fast pattern matching, in-memory state, strict per-stage time budgets, early exit

2. Key Terms (Glossary)

These words appear throughout the document. Each is explained in one or two plain sentences.

Term	Simple meaning
HTTP / HTTPS	The language browsers and apps use to ask websites for things. HTTPS is the encrypted version.
Request / Response	A request is the question (“give me this page”); the response is the answer (the page).
PoP (Point of Presence)	A small data center in a city that contains our servers. Users connect to the nearest one.
Edge	The part of the network closest to users – where our PoPs live.
Origin	The customer’s real web servers – where the website actually lives.
WAF	Web Application Firewall. Software that reads each web request and blocks known attack patterns such as SQL injection or cross-site scripting (XSS).
CDN	Content Delivery Network. Keeps copies (a cache) of content in PoPs so users get it quickly.
DDoS	Distributed Denial of Service. Flooding a site with junk traffic from many machines to knock it offline.
L3 / L4 / L7	Layers of networking. L3/L4 = raw packets and connections (IP, TCP, UDP). L7 = the application content (HTTP headers, URLs, bodies).
Anycast	Announcing the same IP address from many PoPs. The internet automatically delivers a user to the nearest one.
BGP	Border Gateway Protocol – the system internet routers use to tell each other which addresses they can reach.
TLS	The encryption behind HTTPS. “TLS termination” means decrypting traffic so it can be inspected.
Latency	Delay – how long something takes. Measured here in milliseconds (ms). 1 ms = 1/1000 of a second.
p50 / p99	Percentiles. p50 = the typical request. p99 = the slowest 1 in 100 requests. Good systems care about p99.
Control plane / Data plane	Data plane = the machinery that handles live traffic. Control plane = the machinery that configures it (rules, settings).
Canary rollout	Releasing a change to a tiny slice first (like a canary in a coal mine) before everywhere else.
False positive	Blocking a good user by mistake. The WAF must keep these very low.
OWASP CRS	The OWASP Core Rule Set – a widely used open set of WAF rules for common web attacks.
eBPF / XDP	Linux technologies that let tiny, safe programs inspect and drop packets inside the network card driver, before the operating system spends effort on them.

Term	Simple meaning
SLO / SLA	Service Level Objective (internal target) / Service Level Agreement (promise to customers, often with refunds if broken).

3. Architecture Diagram

The architecture is shown at two zoom levels. Figure 1 is the simple picture. Figure 2 is the detailed picture with every component. Figure 3 zooms into how rule changes travel to the PoPs.

3.1 High-level overview

The big picture: every request passes a security checkpoint at the edge before reaching the website

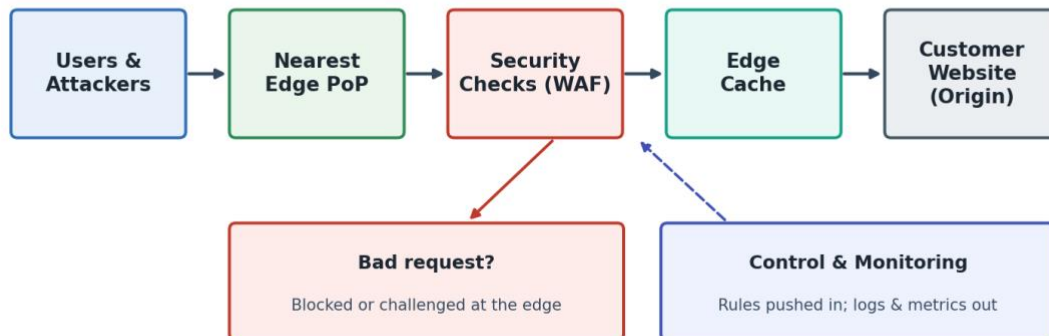


Figure 1 – The big picture: every request is checked at the edge before it reaches the website

How to read Figure 1: read left to right. A user (or an attacker) sends a request. It lands on the nearest Edge PoP. The PoP runs security checks. Bad requests go down the red arrow and are blocked or challenged. Good requests continue to the Edge Cache; if the answer is already stored it is returned immediately, otherwise the request continues to the customer’s website (the origin). The dashed blue arrow shows that the Control & Monitoring systems constantly feed new rules into the security checks and collect logs and metrics back out.

3.2 Detailed architecture

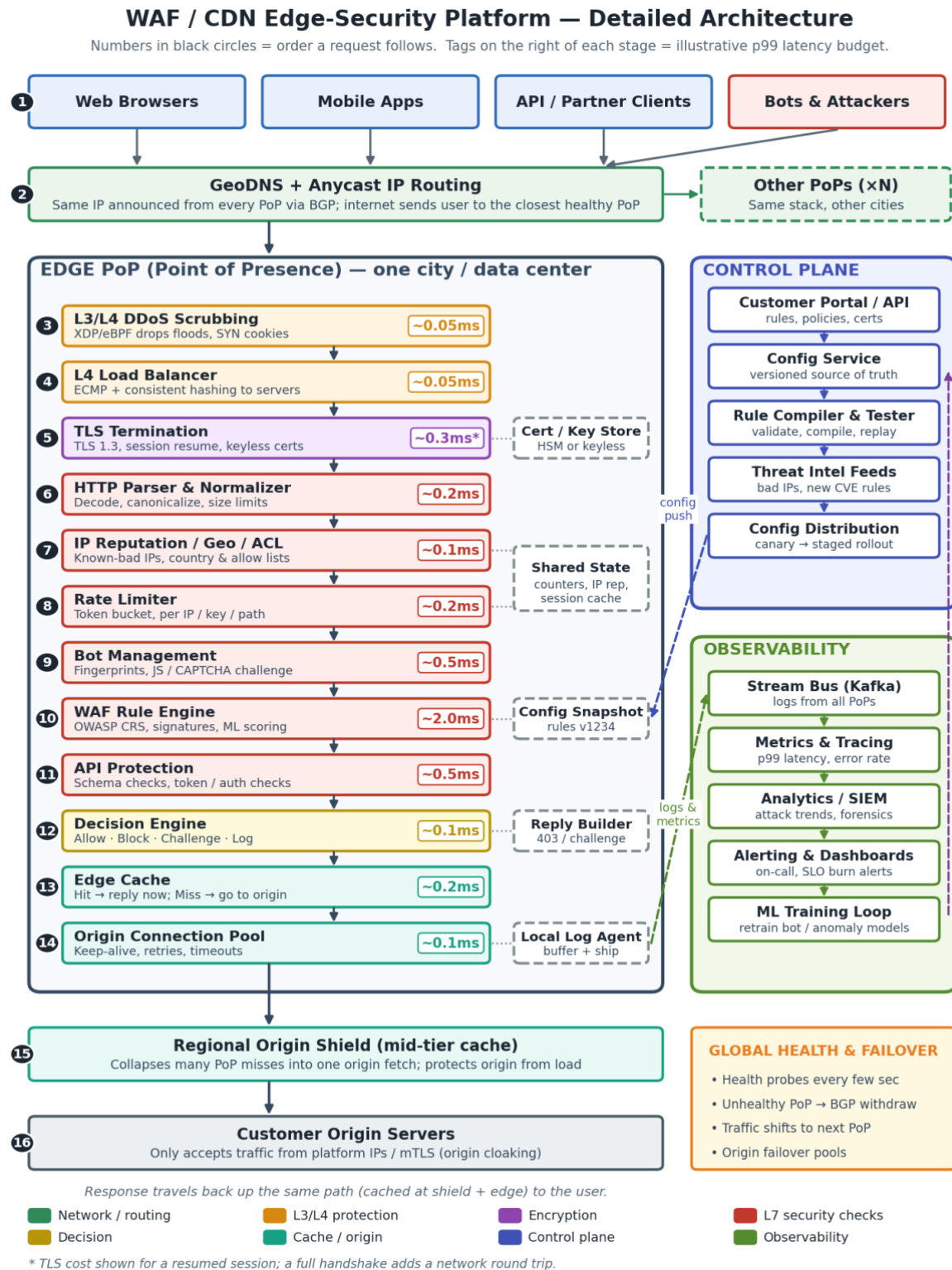


Figure 2 – Detailed end-to-end architecture of the edge-security platform

3.3 Reading the detailed diagram, element by element

Visual conventions

Visual element	What it means
Solid box	A real running component that handles live traffic.
Dashed grey box (inside the PoP)	A helper that the main pipeline consults (key store, shared state, config snapshot, reply builder, log agent). It does not sit in the request path itself.
Dotted grey line	The pipeline stage on the left talks to the helper box on the right.
Solid dark arrow	The live request moving forward to the next step.
Dashed blue arrow “config push”	New rules and settings flowing from the Control Plane into each PoP.
Dashed green arrow “logs & metrics”	Telemetry flowing out of each PoP into the Observability system.
Dashed purple arrow (far right)	The learning loop: what we learn from traffic (new bot patterns, retrained models) goes back into the Config Service as new rules.
Black numbered circle	The order in which a request passes through the system (1 to 16).
Coloured tag on the right of a stage (e.g. ~2.0ms)	The illustrative p99 time budget allowed for that stage.
Box colours	Group the job type: green = network/routing, orange = L3/L4 protection, purple = encryption, red = L7 security checks, yellow = decision, teal = cache/origin, blue = control plane, olive = observability. The legend at the bottom repeats this.

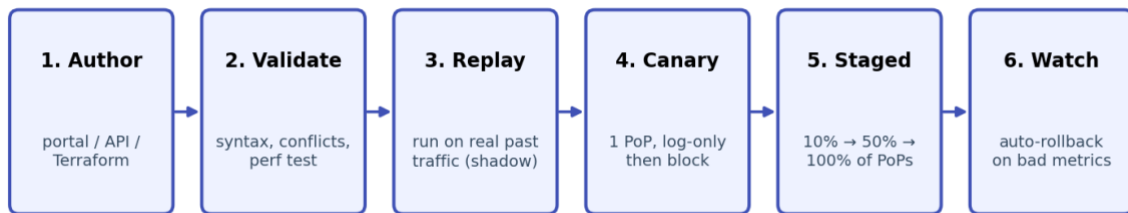
Walking the diagram from top to bottom

#	Element in the diagram	What happens there (simple terms)
1	Web Browsers, Mobile Apps, API / Partner Clients, Bots & Attackers	Everyone who sends traffic. Good users and attackers look the same at first – the platform must tell them apart. Attackers are drawn in red. All four arrows go into the same routing layer.
2	GeoDNS + Anycast IP Routing	Turns the website name into an IP address and makes sure the user is carried to the closest healthy PoP. The arrow to “Other PoPs (xN)” shows there are many identical PoPs in other cities.
3	L3/L4 DDoS Scrubbing	The first filter. Drops junk floods of packets at the network card before they can use any real CPU.
4	L4 Load Balancer	Spreads connections evenly across the many servers inside the PoP.
5	TLS Termination (+ Cert / Key Store)	Decrypts HTTPS so the content can be inspected. Uses certificates from the key store.
6	HTTP Parser & Normalizer	Reads the request and cleans it into one standard form so attackers cannot hide tricks with odd encodings.

#	Element in the diagram	What happens there (simple terms)
7	IP Reputation / Geo / ACL (+ Shared State)	Checks who is asking: known-bad IPs, blocked countries, allow/deny lists.
8	Rate Limiter (+ Shared State)	Counts how often each client is asking and stops anyone going too fast.
9	Bot Management	Decides whether the client is a human, a good bot (like a search engine) or a bad bot.
10	WAF Rule Engine (+ Config Snapshot)	The main attack detector. Looks inside URLs, headers and bodies for attack patterns. The rules come from the config snapshot.
11	API Protection	For API traffic: checks that the request matches the published API shape and carries valid tokens.
12	Decision Engine (+ Reply Builder)	Adds up all the signals and chooses: allow, block, challenge or just log. If blocked, the reply builder sends a 403 page or a challenge page.
13	Edge Cache	If a saved copy of the answer exists, send it straight back. Otherwise continue.
14	Origin Connection Pool (+ Local Log Agent)	Reuses already-open, warm connections to fetch from the origin. The log agent records the request details and ships them out.
15	Regional Origin Shield	A second, bigger cache layer shared by many PoPs so the origin is asked only once for popular content.
16	Customer Origin Servers	The real website. It is locked down so it only accepts traffic coming through the platform.
—	Control Plane (right, blue)	Where customers and engineers change rules. Changes are validated, tested, and rolled out gradually to every PoP.
—	Observability (right, green)	Collects every PoP's logs and metrics, powers dashboards, alerts, security analytics and ML model training.
—	Global Health & Failover (bottom right, orange)	Constantly checks every PoP and origin. If one fails, traffic is automatically moved elsewhere.

3.4 Control-plane rollout diagram

How a rule change safely reaches every PoP (control-plane workflow)



Typical end-to-end time: seconds for urgent threat rules, minutes to hours for normal changes.

Figure 3 – How a rule change travels safely from an engineer to every PoP

How to read Figure 3: a change is written (1), automatically checked for mistakes (2), replayed against real recent traffic to see what it would have blocked (3), switched on in one PoP first – usually in “log-only” mode (4), expanded gradually to more PoPs (5), and watched by automated monitors that can roll it back on their own (6). This is explained fully in Section 5.7.

4. Components Explained in Detail

Each component below follows the same pattern: a plain-language summary, what it is, why it is needed, how it works step by step, the small details that matter, and what happens if it breaks. They are listed in the order a request meets them, followed by the supporting systems.

4.1 Traffic entry

4.1.1 Clients (users, apps, APIs, bots)

In simple terms

Everyone and everything that sends requests to a protected website: people on browsers, mobile apps, other companies' servers calling APIs, good bots such as search engines, and bad bots or attackers.

What it is: The sources of traffic. The platform does not control them and cannot trust them.

Why it is needed: Understanding the mix of clients shapes every design decision. A normal minute might be mostly humans; during an attack the majority can be hostile.

How it works, step by step:

1. A client looks up the website name (for example shop.example.com) via DNS.
2. It opens a network connection to the IP address it receives.
3. It performs a TLS handshake (for HTTPS) and then sends one or more HTTP requests over that connection.
4. It waits for responses; modern clients reuse the connection for many requests (HTTP/2, HTTP/3).

Minute details that matter:

- Protocols supported: HTTP/1.1, HTTP/2 (many requests on one connection), and HTTP/3 over QUIC (runs on UDP, faster on mobile networks).
- Good bots (Google, Bing, uptime monitors) are identified by verified IP ranges and reverse-DNS checks so they are never blocked by mistake.
- Attackers deliberately imitate real browsers, rotate IP addresses and spread requests thinly to avoid detection.

4.1.2 GeoDNS and Anycast routing

In simple terms

This is the “signpost” system. It makes sure every user is sent to the closest working PoP, automatically, without the user doing anything.

What it is: Two complementary techniques. **GeoDNS** answers DNS lookups with an address chosen by the user's location. **Anycast** announces the very same IP address from every PoP using BGP, so internet routers naturally deliver packets to the nearest one.

Why it is needed: Distance equals delay: light in fibre travels about 200 km per millisecond, so a user 5,000 km away pays roughly 50 ms round trip before any work starts. Sending users to a nearby PoP removes most of that. Anycast also spreads a DDoS attack across all PoPs instead of focusing it on one.

How it works, step by step:

1. The customer points their domain (via a CNAME or by using the platform's name servers) at the platform.
2. The platform's authoritative DNS servers – themselves anycast and spread worldwide – answer the lookup.
3. The answer is an anycast IP. Every PoP announces that IP range to its internet neighbours with BGP.
4. Internet routers pick the shortest BGP path, which is usually the geographically nearest PoP.
5. If a PoP stops announcing (because it is unhealthy or under maintenance), routers automatically pick the next best PoP within seconds.

Minute details that matter:

- DNS answers use short TTLs (for example 30–300 seconds) so changes take effect quickly.
- Traffic engineering: BGP settings such as path prepending and community tags let operators nudge traffic away from a busy PoP.
- Anycast is well suited to short connections; for long connections the design makes sure route changes are rare, and clients simply reconnect if one happens.

If it breaks: If DNS failed, nobody could find the sites, so DNS is itself anycast across every PoP with multiple independent providers as backup.

4.2 Inside the Edge PoP – the data plane

4.2.1 Edge PoP (the container)

In simple terms

A small, self-contained data center in one city with racks of identical servers that can do the whole job – filtering, decrypting, checking, caching and forwarding – on their own.

What it is: A physical site made of routers, switches and a fleet of general-purpose servers. Every server runs the same software stack, so any server can handle any request.

Why it is needed: Having many independent PoPs means one failure affects only a small slice, and users everywhere get low latency.

How it works, step by step:

1. Border routers receive traffic from internet providers and exchanges.
2. Traffic passes through the PoP's filtering and load-balancing layers to a server.
3. That server runs the full L7 pipeline (Sections 4.2.5 – 4.2.13) in one process, avoiding slow network hops between services.

- Results, logs and metrics are shipped out asynchronously so they never slow down the request.

Minute details that matter:

- Design principle: **each PoP can run for a while even if cut off from the control plane** (it keeps using its last known good configuration).
- Capacity is planned so that a PoP can absorb the traffic of a failed neighbour (N+1 or N+2 headroom).
- Servers are “cattle, not pets”: any server can be removed, re-imaged or replaced without customer impact.

4.2.2 L3/L4 DDoS scrubbing

In simple terms

A very fast bouncer at the door that throws out obvious junk – floods of fake packets – before they reach the more expensive checks.

What it is: Packet-level filtering that runs as early as possible, ideally inside the network card driver using **XDP/eBPF**, plus router-level filters for the very biggest floods.

Why it is needed: Volumetric attacks can reach terabits per second or hundreds of millions of packets per second. If each packet reached normal software, the servers would drown. Dropping them at the earliest point costs almost nothing.

How it works, step by step:

- Each packet arrives at the network card.
- A small eBPF program inspects the headers (source IP, ports, flags, packet size) in nanoseconds.
- It drops packets that match active attack fingerprints (for example, UDP floods to a web port, malformed packets, reflection traffic from DNS/NTP/memcached servers).
- For TCP SYN floods, it uses **SYN cookies**, so the server does not store any state until the client proves it is real by completing the handshake.
- Remaining packets pass to the load balancer. Counters of dropped traffic are exported to monitoring.

Minute details that matter:

- Fingerprints are generated automatically: a detector samples traffic, spots unusual patterns, and pushes a new drop rule to all servers in seconds.
- Per-source and per-destination packet rate limits stop any single IP from overwhelming a server.
- Because of anycast, a global attack is split across all PoPs, so each PoP only sees a fraction.

If it breaks: If the scrubbing layer overloads, the router can fall back to coarse filtering, and BGP can shift some traffic to PoPs with more spare capacity.

Latency budget: ~0.05 ms (effectively free).

4.2.3 L4 load balancer

In simple terms

A traffic officer inside the PoP who spreads incoming connections evenly across all the servers and keeps each connection going to the same server.

What it is: A layer-4 (TCP/UDP) load balancer, usually built from router **ECMP** (equal-cost multi-path) plus a software balancer that uses **consistent hashing**.

Why it is needed: Without it, some servers would be overloaded while others sit idle. It also allows servers to be removed for maintenance without dropping users.

How it works, step by step:

1. The router hashes each packet's connection details (source IP/port, destination IP/port) and sends it to one of the load-balancer servers.
2. The software balancer uses consistent hashing to pick a backend server, so the same connection always lands on the same server.
3. Health checks run every second or so; unhealthy servers are removed from the hash ring.
4. When servers are added or removed, consistent hashing means only a small share of connections move.

Minute details that matter:

- Connection tracking tables handle the rare case where the ring changes mid-connection.
- Direct Server Return (DSR) can be used so replies skip the balancer and go straight back, saving capacity.
- QUIC/HTTP3 uses connection IDs instead of IP/port so that mobile users switching networks stay on the same server.

Latency budget: ~0.05 ms.

4.2.4 TLS termination and certificate/key store

In simple terms

The locked envelope (HTTPS) is opened here so the guards can read what is inside. The keys to open it are kept extremely safe.

What it is: The step where the platform completes the encrypted handshake with the client and decrypts the request. Certificates and private keys come from a secured store (a hardware security module or a “keyless” service).

Why it is needed: The WAF cannot inspect encrypted traffic. Decrypting at the edge is also faster for users because the handshake happens nearby.

How it works, step by step:

1. The client sends a “ClientHello” including the website name (SNI).
2. The server looks up the matching certificate for that name in memory.

3. It completes a TLS 1.3 handshake (one round trip), or a resumed handshake (zero or one round trip) for returning clients using session tickets.
4. The private-key operation is done either locally with keys held in protected memory, or remotely via a keyless signing service so that sensitive customer keys never leave the customer's or the platform's secure vault.
5. Decrypted HTTP now flows into the parser. Replies are encrypted again on the way out.

Minute details that matter:

- Modern ciphers only (for example AES-GCM, ChaCha20-Poly1305); old protocols like SSL 3 and TLS 1.0 are disabled or restricted.
- Elliptic-curve certificates are preferred because signing is much cheaper than with RSA.
- Session resumption and HTTP/2 connection reuse mean most requests do not pay any handshake cost at all.
- Certificates are issued and renewed automatically (for example via ACME) and distributed through the control plane.
- TLS fingerprints (such as JA3/JA4) are recorded here and later used by bot management.

If it breaks: If the key service is unreachable, PoPs use cached session keys and fail over to another key-service region.

Latency budget: ~0.3 ms CPU for a resumed session; a full handshake adds one network round trip, which is why resumption matters.

4.2.5 HTTP parser and normalizer

In simple terms

Reads the request carefully and rewrites it into one standard, clean form, so attackers cannot sneak an attack past the guards by writing it in a strange way.

What it is: A strict, high-performance HTTP parser plus a normalization step (also called canonicalization).

Why it is needed: Attackers disguise payloads using double URL-encoding, mixed case, Unicode tricks, comments inside SQL, or path tricks such as `"/././"`. If the WAF checks the disguised version, the attack slips through. Also, ambiguous requests can cause "request smuggling", where the edge and the origin disagree on where one request ends.

How it works, step by step:

1. Parse the request line, headers and body according to the HTTP standard, rejecting anything malformed.
2. Enforce limits: maximum URL length, header count, header size, body size, and time allowed to receive the request (stops "slowloris" slow attacks).
3. Decode layers in a safe order: URL-decoding, HTML entities, Unicode normalization, base64 where relevant, lower-casing where safe.
4. Resolve path tricks (`"/a/./b/./c"` becomes `"/a/c"`) and remove null bytes.

5. Parse bodies by type – form data, JSON, XML, multipart uploads – into fields the WAF can inspect individually.
6. Reject ambiguous requests (for example conflicting Content-Length and Transfer-Encoding headers) to prevent request smuggling.

Minute details that matter:

- Only the first N kilobytes of large bodies are inspected in line; very large uploads are streamed with lighter checks to protect the latency budget.
- Both the original and normalized values are kept so rules can use either and logs show exactly what was received.
- The parser is fuzz-tested heavily because a parsing bug here is a security bug.

Latency budget: ~0.2 ms.

4.2.6 IP reputation, geo and access-control lists (ACL)

In simple terms

Checks who is knocking before looking at what they are carrying: is this address known to be bad, is it from a blocked country, or is it on the customer's allow or deny list?

What it is: Fast lookups against several lists: global IP reputation scores, geolocation data, customer allow/deny lists, anonymizer/Tor/VPN lists, and cloud-hosting ranges.

Why it is needed: It is the cheapest L7 check. Stopping well-known bad sources early saves the cost of deeper inspection.

How it works, step by step:

1. Take the client IP (and, when behind a trusted proxy, the real client IP from a verified header).
2. Look it up in an in-memory prefix tree (radix trie) that holds millions of IP ranges.
3. Attach attributes to the request: country, network owner (ASN), reputation score, list memberships.
4. Apply customer rules: allow-list entries skip later checks; deny-list entries are blocked immediately.
5. Pass the rest onward, carrying the reputation score as a signal for the decision engine.

Minute details that matter:

- Reputation data is refreshed every few minutes from the threat-intelligence pipeline.
- Scores decay over time so an IP that was once bad (for example a reused mobile address) is not blocked forever.
- IPv6 is handled by scoring whole prefixes, because attackers can rotate through billions of IPv6 addresses.

Latency budget: ~0.1 ms.

4.2.7 Rate limiter

In simple terms

Counts how many requests each client is making and stops anyone who is going too fast – like a turnstile that only lets a certain number through per minute.

What it is: A counting system that applies limits per IP, per API key, per user session, per URL path, or combinations of these.

Why it is needed: Stops brute-force password guessing, credential stuffing, scraping and application-level (L7) DDoS where every single request looks valid but there are far too many.

How it works, step by step:

1. Build a key for the request, for example “IP + /login”.
2. Look up the counter for that key in memory using a **token bucket** or **sliding window** algorithm.
3. If tokens remain, take one and continue. If not, mark the request as over the limit.
4. Periodically share counters with other servers in the PoP (and, for global limits, across PoPs) using a fast shared store or gossip protocol.
5. The decision engine then blocks, delays, or challenges requests that are over the limit.

Minute details that matter:

- Counters are kept locally and synced in small batches, so the check itself never waits on the network. The trade-off is that limits are “approximately exact” – slightly over the limit for a moment is accepted in exchange for speed.
- Limits can be set in plain terms by customers: “100 requests per minute per IP on /login”.
- Responses include standard headers (such as Retry-After) so well-behaved clients slow down on their own.

Latency budget: ~0.2 ms.

4.2.8 Bot management

In simple terms

Works out whether the visitor is a real human, a helpful robot (like Google’s crawler), or a harmful robot – and treats each one appropriately.

What it is: A scoring system that combines many small signals into a “bot score”, plus challenge mechanisms to test suspicious clients.

Why it is needed: A large share of internet traffic is automated. Bad bots scrape prices, hoard tickets or sneakers, test stolen passwords and create fake accounts – all with requests that look perfectly normal individually.

How it works, step by step:

1. Collect signals: TLS fingerprint, HTTP/2 settings, header order, user-agent consistency, IP reputation, request timing, and whether the client has run the platform’s JavaScript before.

2. A machine-learning model (small and fast, running in memory) turns these signals into a score, for example 1 (certainly bot) to 99 (certainly human).
3. Verified good bots are recognized and allowed.
4. For uncertain scores, the decision engine can issue a **challenge**: an invisible JavaScript check, a proof-of-work puzzle, or, as a last resort, a CAPTCHA.
5. A client that passes receives a signed, short-lived cookie, so it is not challenged again on every request.

Minute details that matter:

- Signals that disagree are strong evidence: a client claiming to be Chrome on Windows but with a TLS fingerprint of a Python script is almost certainly a bot.
- Models are retrained regularly from labelled traffic (Section 5.9) and rolled out like any other rule.
- Customers choose the action per score band and per path, for example strict on /checkout, relaxed on /blog.

Latency budget: ~0.5 ms.

4.2.9 WAF rule engine

In simple terms

The heart of the security system. It looks inside every request for the signatures of known attacks – like a guard who knows exactly what a weapon looks like in an X-ray.

What it is: An engine that evaluates thousands of rules against every part of the request: URL, query string, headers, cookies and body fields. Rules come from the OWASP Core Rule Set, the platform's own managed rules, virtual patches for new vulnerabilities (CVEs), and each customer's custom rules.

Why it is needed: It stops application attacks such as SQL injection, cross-site scripting (XSS), remote code execution, path traversal, local/remote file inclusion, server-side request forgery (SSRF) and exploitation of specific software bugs – attacks the lower layers cannot see.

How it works, step by step:

1. Receive the normalized request and the configuration snapshot for this customer.
2. Pre-filter: all rule patterns are compiled together into a combined matcher (for example with Hyperscan or an Aho-Corasick automaton). One pass over the text finds every rule that might match.
3. Only rules whose pre-filter hit are then checked fully (regular expressions, logic, conditions). This avoids running thousands of expensive regexes on every request.
4. Apply specialised detectors, such as a SQL/JS tokenizer (libinjection-style) that recognises code-like structure rather than just keywords.
5. Add up an **anomaly score**: each matching rule adds points based on severity. The ML anomaly model can add its own score.
6. Hand the total score and the list of matched rules to the decision engine.

Minute details that matter:

- **Paranoia levels** let customers trade stricter protection against more false positives.
- **Exclusions** stop specific rules on specific fields (for example, allow HTML in a blog editor's body field).
- Every rule has a performance test; regular expressions are checked for catastrophic backtracking (ReDoS) and use engines with guaranteed linear time.
- A hard time limit per request: if inspection would exceed its budget, the engine stops and applies the customer's chosen fail-open or fail-closed policy, and the event is logged for investigation.
- New rules always start in **log-only mode** so their real-world false-positive rate is known before they block anything.

If it breaks: A crashing rule can never take the PoP down: rules are sandboxed, compiled and tested before release, and a bad release is automatically rolled back.

Latency budget: ~2.0 ms – the largest share of the budget, because this is the deepest inspection.

4.2.10 API protection**In simple terms**

For apps that talk to each other through APIs, this checks that every request follows the official rules of that API – right shape, right fields, valid login token.

What it is: Validation of API requests against a schema (such as OpenAPI/Swagger), plus token checks (for example JSON Web Tokens), mTLS client certificates and per-endpoint limits.

Why it is needed: APIs are a major attack target. Attacks often use valid-looking requests with extra fields, wrong types, or someone else's record IDs. A positive security model ("only allow what is expected") catches these far better than attack signatures alone.

How it works, step by step:

1. Match the request to a known API endpoint (method + path template, for example POST /v1/orders).
2. Validate path, query and body against the schema: required fields, types, lengths, allowed values.
3. Validate the authentication token: signature, expiry, issuer, audience – using cached public keys so no network call is needed.
4. Apply endpoint-specific rate limits and sensitive-data rules (for example flag responses that look like bulk personal data).
5. Report unknown or "shadow" endpoints that are not in the schema, which often reveals forgotten APIs.

Minute details that matter:

- Schemas can be uploaded by the customer or learned automatically from observed traffic.
- Validation code is generated/compiled from the schema ahead of time for speed.

Latency budget: ~0.5 ms (only for API traffic).

4.2.11 Decision engine and reply builder

In simple terms

The judge. It listens to every guard's opinion and makes one final decision: let it in, block it, test it with a challenge, or let it in but write it down.

What it is: A small policy evaluator that combines all signals – reputation, rate-limit status, bot score, WAF anomaly score, API validation result and customer policies – into one action. The reply builder produces the blocked or challenge page.

Why it is needed: Separating “detection” from “decision” keeps things flexible: the same signals can lead to different actions for different customers or different parts of a site.

How it works, step by step:

1. Collect all signals attached to the request by earlier stages.
2. Evaluate the customer's policy in priority order (for example: allow-list first, then hard blocks, then scores against thresholds).
3. Choose an action: **Allow**, **Block** (HTTP 403), **Challenge** (JS/CAPTCHA page), **Rate-limit** (HTTP 429), or **Log only**.
4. For block or challenge, the reply builder returns a branded page including a reference ID the user can quote to support.
5. For allow, the request continues to the cache. Either way, the decision and reasons are sent to the log agent.

Minute details that matter:

- Early exit: a hard block from an early stage (such as a deny-listed IP) skips the remaining checks, saving time.
- Decisions are deterministic and explainable: every log line lists exactly which rules and scores caused the outcome.
- Customers can run a policy in “simulate” mode to see what it would do before turning it on.

Latency budget: ~0.1 ms.

4.2.12 Edge cache (CDN)

In simple terms

A storage room at the PoP that keeps copies of popular files. If the answer is already here, the user gets it immediately without bothering the real website.

What it is: A two-tier cache on each server (hot items in RAM, larger items on fast SSD), shared logically across the PoP's servers.

Why it is needed: Serving from cache is much faster for users and reduces load on the customer's servers – often 80–95% of static traffic never reaches the origin.

How it works, step by step:

1. Build a cache key (by default: host + path + query string; customers can add or remove parts).
2. Look it up. On a **hit**, check freshness (Cache-Control, max-age). If fresh, return it immediately.
3. On a **miss** or stale item, forward the request toward the origin shield/origin.
4. When the response arrives, store it if it is cacheable, then return it to the user.
5. Evict old or unpopular items when space runs low (LRU-style policies).

Minute details that matter:

- Security runs **before** the cache, so even cached content is protected from abusive clients.
- Request collapsing: if 1,000 users ask for the same missing item at the same moment, only one request goes upstream; the rest wait for that answer.
- Stale-while-revalidate and stale-if-error let the edge keep serving slightly old content if the origin is slow or down – a big availability win.
- Instant purge: customers can remove content from every PoP worldwide in seconds through the control plane.
- Personal or private responses (with cookies/authorization) are not cached unless explicitly configured.

Latency budget: ~0.2 ms for a lookup.

4.2.13 Origin connection pool**In simple terms**

Keeps a set of already-open, ready-to-use connections to the customer's servers, so each request does not waste time setting up a new connection.

What it is: A pool of persistent (keep-alive) TLS connections from the PoP to the origin shield and origin, with retry, timeout and circuit-breaker logic.

Why it is needed: Opening a new encrypted connection across the world can cost several round trips (100+ ms). Reusing warm connections removes that cost.

How it works, step by step:

1. Pick an existing idle connection to the right origin (or open one if needed).
2. Send the request, adding headers such as the real client IP and a request ID for tracing.
3. Apply timeouts (connect, first-byte, total).
4. Retry safely on another origin server for idempotent requests (like GET) if one fails.
5. If an origin keeps failing, the circuit breaker stops sending traffic to it for a short time and uses a backup origin or stale cache.

Minute details that matter:

- Traffic to the origin is authenticated (mutual TLS or a signed header) so the origin can reject anyone who is not the platform.
- Load balancing across multiple origin servers with health checks and weights.

Latency budget: ~0.1 ms of CPU; the network time to the origin is not part of the security budget.

4.2.14 Shared state store (inside the PoP)

In simple terms

A shared notebook that all servers in a PoP can read and write very quickly – used for counters, reputation lists and session information.

What it is: An in-memory, low-latency key-value store (for example a local memory cache plus a PoP-level cluster), holding rate-limit counters, challenge-pass tokens, IP reputation tables and TLS session caches.

Why it is needed: Security decisions often need information that one server alone does not have (for example, how many requests this IP sent to all servers in the last minute).

How it works, step by step:

1. Each server keeps a local copy of hot data in its own memory.
2. Updates are batched and sent to the PoP store in the background.
3. Reads almost always come from local memory; the store is consulted only for less common keys.

Minute details that matter:

- Built so that the request path never waits on it: if it is slow or down, servers continue using local data (fail-soft).

4.3 Upstream of the PoP

4.3.1 Regional origin shield (mid-tier cache)

In simple terms

A bigger, regional storage room between the PoPs and the customer's website, so that even when many PoPs miss, the website is only asked once.

What it is: A designated set of larger PoPs (for example one per region) that sit between edge PoPs and the origin.

Why it is needed: With hundreds of PoPs, a popular new file could otherwise cause hundreds of simultaneous requests to the origin. The shield turns that into one.

How it works, step by step:

1. An edge PoP misses in its own cache and asks the shield.
2. The shield checks its (larger) cache and collapses duplicate requests.
3. Only on a shield miss does the request go to the origin.
4. The response is cached at the shield and at the edge on the way back.

Minute details that matter:

- The shield also uses long-lived, optimised connections to the origin across the platform's private backbone, which is often faster than the public internet.

4.3.2 Customer origin servers

In simple terms

The customer's real website or API. The goal is that it only ever sees clean, legitimate traffic.

What it is: The customer's own infrastructure (cloud, data center, or hosting).

Why it is needed: It is the thing being protected.

How it works, step by step:

1. The origin is configured to accept connections only from the platform's IP ranges or with the platform's client certificate (**origin cloaking**).
2. It receives clean requests with extra headers (real client IP, bot score, request ID) it can use for its own logic.
3. It replies; the reply travels back through the shield and edge caches to the user.

Minute details that matter:

- If attackers find the origin's real IP and it is not locked down, they could bypass the WAF – origin lock-down is therefore a required part of onboarding.
- Multiple origins and failover pools can be configured for high availability.

4.4 Control plane (how the platform is configured)

The control plane never touches live requests. Its job is to let customers and engineers change behaviour safely, and to deliver those changes to hundreds of PoPs quickly and without breaking anything.

4.4.1 Customer portal and API

In simple terms

The website and API customers use to set up their protection – add domains, choose rules, set limits, upload certificates, and see what was blocked.

What it is: A web dashboard, a REST/GraphQL API and infrastructure-as-code support (for example a Terraform provider).

Why it is needed: Customers need self-service control, and large customers want to manage security settings in code alongside the rest of their systems.

How it works, step by step:

1. User logs in with strong authentication (SSO, multi-factor) and role-based permissions.
2. They make a change; the API validates the input format immediately.
3. The change is saved as a new version in the config service, with an audit log entry of who changed what and when.

Minute details that matter:

- Every change is versioned and can be rolled back in one click.

- Destructive actions (for example turning the WAF off) require extra confirmation.

4.4.2 Config service (source of truth)

In simple terms

The master rule book. It stores every customer's current settings, plus every previous version.

What it is: A strongly-consistent, replicated database (in several regions) that holds zones, rules, policies, certificates and their version history.

Why it is needed: There must be exactly one correct answer to “what is the configuration right now?”, and it must survive the loss of a whole region.

How it works, step by step:

1. Receives validated changes from the portal/API and from automated sources (threat intel, ML loop).
2. Assigns each change a monotonically increasing version number.
3. Notifies the rule compiler and distribution system that a new version exists.

Minute details that matter:

- Written to rarely, read to by the distribution system; it is never on the live request path, so its latency does not affect users.

4.4.3 Rule compiler and tester

In simple terms

Turns human-friendly rules into a super-fast machine format and tests them thoroughly before they are allowed anywhere near real traffic.

What it is: A build pipeline that validates, compiles and tests configurations.

Why it is needed: Evaluating rules must be very fast at the edge, and a single bad rule (for example a regex that takes seconds) could slow down or break every PoP. Most large outages in edge platforms have come from bad configuration pushes, so this step is critical.

How it works, step by step:

1. Check syntax and logical conflicts (for example an allow rule that makes a block rule unreachable).
2. Check performance: every regex is tested against worst-case inputs; rules that exceed the time budget are rejected.
3. Compile all patterns into a combined matcher database and generate a compact binary config bundle.
4. **Replay test:** run the new bundle against a sample of recent real traffic and compare decisions with the current version; large unexpected changes in block rates stop the release.
5. Sign the bundle cryptographically so PoPs can verify it has not been tampered with.

Minute details that matter:

- The same engine code is used in the tester and at the edge, so test results are trustworthy.

4.4.4 Threat intelligence feeds**In simple terms**

News from the security world – lists of bad IP addresses, newly discovered software vulnerabilities, and attack patterns seen elsewhere – turned into protection automatically.

What it is: Pipelines that gather data from the platform’s own global traffic (the biggest source), security researchers, public vulnerability databases (CVEs) and partner feeds.

Why it is needed: Seeing attacks across thousands of customers means one customer’s attack can protect all the others within minutes.

How it works, step by step:

1. Collect and deduplicate indicators (IPs, fingerprints, payload patterns).
2. Score them for confidence and decide expiry times.
3. For new vulnerabilities, security researchers write a **virtual patch** rule that blocks exploitation even before customers update their software.
4. Publish updates to the config service as managed rule versions.

4.4.5 Config distribution and the PoP config snapshot**In simple terms**

The delivery service that carries each new rule book to every PoP – carefully, a few at a time – and the local copy each PoP keeps.

What it is: A publish/subscribe distribution system (for example a fan-out tree of relays or a replicated key-value store) that pushes signed config bundles to all PoPs, with staged rollout and automatic rollback.

Why it is needed: Changes must reach hundreds of PoPs in seconds for urgent threats, yet a bad change must never reach all of them at once.

How it works, step by step:

1. Release goes first to a canary PoP (often in log-only mode).
2. Health metrics (errors, latency, CPU, block rate, false-positive reports) are compared with a baseline.
3. If healthy, the release expands in waves: for example 1 PoP → 10% → 50% → 100%.
4. Each server verifies the signature, loads the new bundle alongside the old one, then swaps a pointer atomically – no restart, no dropped requests.
5. The previous version stays in memory so rollback is instant.

Minute details that matter:

- If a PoP loses contact with the control plane it continues running on its last known good snapshot (static stability).
- Urgent changes (such as an emergency virtual patch) use a faster lane with shorter waves but still go through validation.

4.5 Observability (how the platform is watched)

4.5.1 Local log agent

In simple terms

A note-taker on every server that writes down what happened to each request and sends those notes to headquarters without slowing anything down.

What it is: A lightweight agent that collects request logs, security events and metrics from the edge process through shared memory or a local socket.

Why it is needed: Customers need to see what was blocked and why; engineers need data to find problems; the ML loop needs training data.

How it works, step by step:

1. The edge process writes a compact record per request into a memory ring buffer (non-blocking).
2. The agent reads the buffer, batches and compresses records.
3. It ships them to the central stream bus; if the network is down it buffers to disk.
4. Under extreme load (e.g., a huge DDoS) it samples non-security logs so that shipping logs never competes with serving traffic.

Minute details that matter:

- Sensitive fields (passwords, card numbers, auth headers) are masked before leaving the server.

4.5.2 Stream bus (e.g. Kafka)

In simple terms

A giant, reliable conveyor belt that carries logs from every PoP to everyone who needs them.

What it is: A durable, partitioned message stream that receives millions of events per second.

Why it is needed: Decouples producers (PoPs) from consumers (analytics, SIEM, customer log exports, ML), so any consumer can be slow or restarted without losing data.

How it works, step by step:

1. Events are written to topics partitioned by customer/zone.
2. Consumers read at their own pace; data is kept for a set period (for example several days) for replay.

4.5.3 Metrics and tracing

In simple terms

The health dashboard numbers – how fast, how many errors, how busy – and the ability to follow one request through every step.

What it is: Time-series metrics (request rate, p50/p99/p999 latency per stage, error rates, CPU, memory, cache hit ratio, block rate) and distributed tracing with a request ID.

Why it is needed: The 10 ms p99 budget can only be defended if every stage's latency is measured continuously.

How it works, step by step:

1. Each stage records its own duration in a histogram.
2. Histograms are aggregated per server, PoP, region and globally.
3. A sample of requests is fully traced to show where time was spent.

4.5.4 Analytics / SIEM

In simple terms

The detective office: it studies all the security events to find attack campaigns and gives customers reports and searchable logs.

What it is: A columnar analytics database for fast queries, customer-facing security analytics, and exports into customers' own SIEM tools (Splunk, Sentinel, etc.).

Why it is needed: Customers need to investigate incidents and prove compliance; the security team needs to spot new attack trends.

How it works, step by step:

1. Consume events from the stream bus.
2. Aggregate by attack type, source, target and time.
3. Power dashboards, search and scheduled reports; push events to customer SIEMs.

4.5.5 Alerting and dashboards

In simple terms

Alarms that wake up the on-call engineer when something is wrong, before customers notice.

What it is: Rules on metrics and logs tied to the SLOs (availability, latency) and to security signals (sudden spikes in blocks or false positives).

Why it is needed: 99.99% availability means incidents must be detected and fixed in minutes.

How it works, step by step:

1. Define SLOs, for example "99.99% of requests succeed" and "p99 security overhead < 10 ms".

2. Alert on **error-budget burn rate** (how fast we are using up the allowed failures) rather than on single noisy numbers.
3. Route alerts to on-call staff with runbooks; many alerts also trigger automatic mitigation (e.g., rollback).

4.5.6 ML training loop

In simple terms

Learning from yesterday's traffic to be smarter tomorrow – retraining the bot and anomaly models on fresh data.

What it is: An offline pipeline that labels traffic (using challenge outcomes, known attacks, customer feedback) and retrains models.

Why it is needed: Attackers adapt constantly; static rules alone go stale.

How it works, step by step:

1. Collect features and outcomes from logs.
2. Train and evaluate new models offline against held-out data.
3. Package the model like a rule bundle and send it through the same safe rollout (shadow mode first).

4.6 Reliability systems

4.6.1 Global health checking and failover

In simple terms

An always-on system that checks every PoP and every customer origin. If something fails, it quietly moves traffic somewhere healthy.

What it is: Distributed health probes plus automation that controls BGP announcements, DNS answers and origin selection.

Why it is needed: At 99.99% there is no time for humans to react manually to routine failures.

How it works, step by step:

1. Probes from many locations test each PoP (and each server) every few seconds with real HTTP requests.
2. If a server fails, the load balancer removes it (seconds).
3. If a whole PoP is unhealthy or overloaded, it withdraws its anycast BGP routes; internet routers send users to the next nearest PoP automatically.
4. If an origin fails, traffic moves to a backup origin pool, or the edge serves stale cached content.
5. When things recover, traffic is brought back gradually to avoid a sudden surge.

Minute details that matter:

- Safety limits stop the automation from withdrawing too many PoPs at once (which would overload the rest).
- Regular “game days” and chaos tests deliberately break things to prove failover works.

5. Workflows Explained End to End

This section follows real situations through the system step by step. The step numbers in 5.1 match the black circles in Figure 2.

5.1 A normal, legitimate request (the “happy path”)

Example

Priya opens <https://shop.example.com/products/42> on her phone in Mumbai. The shop’s servers are in Frankfurt and are protected by the platform.

Step	What happens	In simple terms
1	Priya’s browser needs the IP for shop.example.com.	Her phone asks “where is this shop?”
2	The platform’s anycast DNS answers with an anycast IP; BGP routes her packets to the Mumbai PoP.	She is sent to the nearest gate – in Mumbai, not Frankfurt.
3	Her TCP/QUIC packets pass the L3/L4 scrubber – nothing suspicious.	The door bouncer lets her in.
4	The PoP load balancer hashes her connection to server #37.	She is assigned a free guard.
5	TLS 1.3 handshake completes (or resumes); the request is decrypted.	Her sealed envelope is opened safely.
6	The parser reads GET /products/42, headers and cookies; normalizes them; all limits OK.	The request is read and tidied into a standard form.
7	IP lookup: Indian mobile network, good reputation, not on any list.	Her ID checks out.
8	Rate limiter: 12 requests in the last minute – well under the limit.	She is not rushing.
9	Bot score 92 (human): she has a valid challenge cookie and a real browser fingerprint.	She behaves like a real person.
10	WAF: pre-filter finds no suspicious patterns; anomaly score 0.	No weapons in her bag.
11	Not an API path – API checks skipped.	Not relevant here.
12	Decision engine: ALLOW. A log record is written.	The judge says “come in”.
13	Edge cache: product image and page are cached – HIT for the images, MISS for the personalised page.	Common items come from the snack stand; the personalised page needs the kitchen.
14	The page request uses a warm pooled connection upstream.	A ready phone line to the shop is used.
15	Origin shield in the region has no copy of this personalised page, so it forwards.	The regional warehouse passes the order on.

Step	What happens	In simple terms
16	Frankfurt origin generates the page; response returns via shield → Mumbai PoP → Priya.	The kitchen cooks the dish and it is carried back.

Where the time goes (illustrative): security checks inside the PoP $\approx$ 1–2 ms typical and under 5 ms at p99; the Mumbai-to-Frankfurt round trip for the uncached page is far larger ($\sim$ 120 ms) and is the part the CDN tries to avoid entirely by caching. Cached items return in a few milliseconds end to end.

5.2 A web attack is blocked (SQL injection)

Example

An attacker sends: **GET**

/search?q=1%27%20UNION%20SELECT%20password%20FROM%20users--

1. Routing, scrubbing, load balancing and TLS work exactly as in 5.1 – the attacker looks like any other client at the network level.
2. The **parser/normalizer** URL-decodes the query: `q = 1' UNION SELECT password FROM users--`. It also lower-cases and removes SQL comments in a copy used for matching.
3. **IP reputation:** the address belongs to a cloud hosting provider and has a medium-risk score – a small signal, not enough to block.
4. **Rate limiter:** within limits (the attacker is going slowly).
5. **Bot management:** TLS fingerprint matches a scripting tool while the user-agent claims to be Chrome – bot score 8 (likely bot).
6. **WAF rule engine:** the combined pre-filter hits on “union”, “select” and a quote character in the same parameter. The full SQL-injection rules and the SQL tokenizer confirm a UNION-based injection. Anomaly score jumps above the block threshold (for example 25 vs threshold 5).
7. **Decision engine:** BLOCK. Reasons: rule “SQLi – UNION attack” + low bot score.
8. **Reply builder:** returns HTTP 403 with a block page and a reference ID.
9. **Log agent** ships the event; the customer sees it in their security dashboard within seconds; the attacker’s IP reputation is raised so repeated attempts are blocked even earlier.
10. The origin never saw the request.

5.3 A suspicious client is challenged (bot management)

1. A client requests `/tickets/concert-123` a few seconds after sales open. Its bot score is 35 – uncertain.
2. The customer policy for `/tickets/*` says: score below 30 = block, 30–60 = challenge, above 60 = allow.
3. The decision engine returns a **challenge page** (HTTP 403/503 with a small JavaScript program) instead of the ticket page.

4. A real browser runs the script invisibly in about a second: it checks browser features and solves a small proof-of-work.
5. The browser sends the answer to a platform endpoint; the PoP verifies it and issues a signed clearance cookie valid for, e.g., 30 minutes.
6. The browser automatically retries the original request with the cookie; the bot score rises and the request is allowed.
7. A simple script cannot run the challenge, so it never gets the cookie and never reaches the ticket page.
8. If the client fails repeatedly, the policy can escalate to a CAPTCHA or a block.

5.4 A large DDoS attack

1. A botnet begins sending 2 Tbps of UDP and SYN traffic at a customer's IP.
2. **Anycast** automatically splits the flood across every PoP that announces the address – each PoP sees only its regional share.
3. In each PoP, sampling detects the sudden change in packet patterns within seconds and generates a fingerprint (for example: UDP to port 443 with a fixed payload size).
4. The fingerprint is pushed to the **XDP/eBPF** programs on every server; packets matching it are dropped in the network driver at line rate.
5. SYN floods are absorbed by **SYN cookies**; real clients complete their handshakes normally.
6. If a PoP's internet links are nearly full, traffic engineering shifts some traffic to PoPs with spare capacity, or upstream providers are asked to filter at their edge.
7. For an **L7 DDoS** (millions of valid-looking HTTP requests), the rate limiter and bot management take over: challenge pages and adaptive rate limits separate the botnet from real users.
8. The customer's origin sees normal traffic throughout. Metrics and a DDoS report are available in the dashboard.

5.5 A client exceeds a rate limit

1. Customer rule: "max 5 POST requests per minute per IP to /login".
2. A credential-stuffing tool sends 200 login attempts per minute from one IP.
3. Requests 1–5 pass and each takes one token from that IP's bucket.
4. Request 6 finds the bucket empty. The rate limiter marks it "over limit".
5. The decision engine returns HTTP 429 "Too Many Requests" with a Retry-After header, or a challenge, as configured.
6. Local counters are synced with the other servers in the PoP within milliseconds, so switching servers does not help the attacker.
7. If the attacker spreads across thousands of IPs, the bot score and login-specific detection (many different usernames, high failure rate) catch the pattern instead.

5.6 Cache hit versus cache miss

Situation	Path taken	Result
Hit at edge	Security checks → edge cache has a fresh copy → reply	Fastest: a few ms end to end; origin not involved.
Miss at edge, hit at shield	Security → edge miss → shield hit → store at edge → reply	Fast; origin still not involved.
Miss everywhere	Security → edge miss → shield miss → origin → store at shield and edge → reply	Slowest, but next user gets a hit.
Stale item, origin down	Security → edge has stale copy → origin fails → serve stale	User still gets content (stale-if-error) – keeps availability high.
1,000 users request the same missing item at once	Request collapsing sends only one request upstream	Origin protected from a sudden stampede.

5.7 A rule change rolling out (control-plane workflow)

See Figure 3 for the picture. Example: the security team writes a virtual patch for a newly announced vulnerability.

1. **Author:** a researcher writes the rule in the rule language and opens a change request. It is peer-reviewed.
2. **Validate:** the compiler checks syntax, conflicts and regex safety; the rule is compiled into the matcher database.
3. **Replay:** the new bundle is run against a sample of the last day's real traffic. It flags the known exploit samples and, crucially, very few normal requests (low false positives).
4. **Canary:** the bundle is deployed to one PoP in **log-only** mode. The team watches what it would have blocked.
5. **Staged rollout:** it expands to 10%, then 50%, then 100% of PoPs – first in log mode, then switched to block mode. Each wave waits for healthy metrics.
6. **Watch:** automated monitors compare errors, latency, CPU and block rates to baseline. Any anomaly triggers an automatic rollback to the previous version in seconds.
7. On every server the new bundle is loaded next to the old one and swapped atomically – no restarts, no dropped connections.
8. The customer-facing changelog is updated; customers can see and, if needed, override the new managed rule.

5.8 A PoP or origin fails (failover)

1. A power problem takes half of the Singapore PoP's servers offline.
2. Within seconds, local load-balancer health checks remove the failed servers; the remaining servers keep serving.

3. Global health probes see rising latency and errors from Singapore. Automation decides the PoP cannot handle its load safely.
4. Singapore withdraws some or all of its anycast BGP routes. Internet routers re-route those users to the next closest PoPs (for example Kuala Lumpur, Jakarta, Hong Kong), usually within seconds to a minute.
5. Those PoPs have headroom planned for exactly this and absorb the extra traffic. Users may see a few extra milliseconds of latency, but no outage.
6. Separately, if a **customer origin** fails, the origin pool fails over to a backup origin; if no backup exists, the edge serves stale cached content where possible and a friendly error page otherwise.
7. When Singapore recovers, routes are re-announced gradually to avoid a sudden surge.

5.9 Telemetry and the learning loop

1. Every request produces a compact log record: time, customer, path, decision, matched rules, bot score, stage latencies.
2. The local log agent batches and compresses records and ships them to the stream bus.
3. Metrics are aggregated in near real time for dashboards and SLO alerting.
4. Security events feed analytics, customer dashboards and customer SIEM exports.
5. Labelled outcomes (challenge passed/failed, confirmed attacks, customer false-positive reports) are used to retrain bot and anomaly models.
6. New IP reputation scores, fingerprints and models are published back to the config service and rolled out safely – closing the loop so the platform gets smarter every day.

5.10 Emergency response to a zero-day vulnerability

1. A critical vulnerability in a popular software library is published, and exploitation begins within hours.
2. The security team analyses exploit samples and writes a virtual-patch rule.
3. The rule goes through the fast lane of the pipeline: validation and replay still happen, but waves are shorter.
4. It reaches all PoPs worldwide within minutes to an hour, protecting every customer running the vulnerable software – even those who have not patched yet.
5. Logs show exploitation attempts being blocked across the network; indicators from these attempts feed IP reputation for all customers.

6. How the Three Headline Targets Are Met

6.1 Handling millions of requests per second

- **Scale out, not up:** capacity grows by adding PoPs and servers; every server is identical and stateless for request handling.
- **Anycast spreads load** globally, including attack load.
- **Cheap work first:** packets are filtered in the kernel before any expensive processing; IP lookups before deep inspection; early exit on hard blocks.
- **Everything in one process** on each server – no network hops between security stages.
- **Efficient code:** a multi-threaded, event-driven server (one worker per CPU core, pinned to cores, no locks on the hot path), zero-copy buffers, and compiled rule sets.
- **Caching** removes a large share of work from the origin and the network.

6.2 Sustaining 99.99%+ availability

Availability	Downtime per year	Downtime per month
99.9%	≈ 8 hours 46 minutes	≈ 43.8 minutes
99.99% (target)	≈ 52.6 minutes	≈ 4.4 minutes
99.999%	≈ 5.3 minutes	≈ 26 seconds

- **No single point of failure:** redundant routers, links, servers, PoPs, DNS providers and control-plane regions.
- **Small blast radius:** one PoP, one server or one customer's config can fail without affecting the others (cell-based design).
- **Static stability:** PoPs keep running on the last good config even if the control plane is down.
- **Safe changes:** canary + staged rollout + automatic rollback for both code and configuration – the biggest single protection, because most outages come from changes.
- **Automatic failover:** server removal in seconds, BGP withdrawal for PoPs, backup origins and stale-cache serving.
- **Graceful degradation:** under extreme load, non-critical work (detailed logs, heavy ML) is shed first so core serving continues.
- **Error budgets:** if the month's allowed downtime is being used up, risky releases pause.

6.3 Keeping security overhead under single-digit milliseconds at p99

Stage	Illustrative p99 budget	Main technique to stay within budget
L3/L4 scrubbing + load balancing	~0.1 ms	Kernel/XDP processing, no per-packet state
TLS (resumed session)	~0.3 ms	TLS 1.3, ECDSA keys, session resumption, connection reuse

Stage	Illustrative p99 budget	Main technique to stay within budget
Parse + normalize	~0.2 ms	Single-pass parser, strict limits, inspect only first N KB of bodies
IP reputation / geo / ACL	~0.1 ms	In-memory radix tree lookups
Rate limiting	~0.2 ms	Local counters, async sync
Bot management	~0.5 ms	Small in-memory model, cached verdicts via clearance cookie
WAF rule engine	~2.0 ms	Combined pre-filter (Hyperscan/Aho-Corasick), linear-time regex, per-request time cap
API validation	~0.5 ms	Schema compiled ahead of time, cached token keys
Decision + cache lookup	~0.3 ms	Simple policy evaluation; RAM-first cache
Total (worst-case sum)	≈ 4–5 ms	Leaves headroom inside the < 10 ms target

Stage p99s do not simply add up in practice (it is rare for one request to be slow at every stage), so the real end-to-end p99 is usually lower than the sum. The headroom protects against noisy neighbours, garbage-collection pauses and traffic spikes.

How the budget is enforced: every stage is timed; releases that raise p99 are automatically rolled back; each rule has a performance test; there is a hard per-request inspection time cap with a customer-chosen fail-open/fail-closed policy; and CPU headroom is monitored so servers never run hot enough to queue requests.

7. Key Design Decisions and Trade-offs

Decision	Choice made	Why	Trade-off accepted
Where to inspect	At the edge, in every PoP	Low latency and attacks stopped near the source	Must run the full stack in hundreds of places
Routing	Anycast + GeoDNS	Automatic nearest-PoP routing and natural DDoS spreading	Less precise control than pure DNS steering
Pipeline shape	All L7 stages in one process per server	No network hops, lowest latency	A bug can affect all stages – mitigated by testing and sandboxing
Rule evaluation	Compiled multi-pattern pre-filter then full rules	Thousands of rules at micro-second speeds	Complex compiler and build pipeline
Rate-limit counters	Local first, synced asynchronously	Never wait on the network	Limits are approximate for a short moment
Config delivery	Signed bundles, staged rollout, atomic swap	Safe, fast, reversible changes	Urgent changes take minutes, not milliseconds
Failure policy	Customer-selectable fail-open or fail-closed	Availability-first sites vs security-first sites differ	Needs clear defaults and documentation
New rules	Log-only first, then block	Keeps false positives very low	Short delay before new protection blocks
Logging	Asynchronous, sampled under extreme load	Logging never slows traffic	Some non-security logs may be sampled during huge attacks

8. Recap: Explaining the Project in Two Minutes

A short spoken version

“I owned the architecture of an edge-security platform – a WAF combined with a CDN – that sits in front of thousands of customer websites and APIs. Traffic reaches the nearest of our many PoPs through anycast. In each PoP we drop network floods in the kernel with eBPF, terminate TLS, normalize the HTTP request and run it through IP reputation, rate limiting, bot scoring, a compiled WAF rule engine and API schema validation. A decision engine then allows, blocks or challenges it, and clean requests are served from cache or forwarded to the origin over warm connections.

The platform handled millions of requests per second. We held availability above 99.99% by removing single points of failure, keeping PoPs statically stable, and shipping every code and config change through canary and staged rollouts with automatic rollback. We kept security overhead in the single-digit milliseconds at p99 by giving each stage a latency budget, compiling thousands of rules into one fast matcher, keeping all state in memory, and measuring every stage’s latency continuously.”

Key takeaways

- **Edge** – do the work close to users, in many PoPs.
- **Layers** – cheap checks first, expensive checks last, with early exit.
- **Speed** – compiled rules, in-memory state, strict per-stage budgets.
- **Safety** – staged, reversible changes; the most common cause of outages is handled by process and automation.
- **Resilience** – every layer can fail and traffic will still flow.
- **Learning** – telemetry feeds back into rules and models every day.